

本小节内容

指针的偏移

指针与一维数组

1 指针的偏移

前面介绍了指针的传递。指针即地址，就像我们找到了一栋楼，这栋楼的楼号是 B，那么往前就是 A，往后就是 C，所以应用指针的另一个场景就是对其进行加减，但对指针进行乘除是没有意义的，就像家庭地址乘以 5 没有意义那样。在工作中，我们把对指针的加减称为指针的偏移，加就是向后偏移，减就是向前偏移。下面我们来看例 1.1。

【例 1.1】指针的偏移使用场景。

```
#include <stdio.h>

#define N 5
//指针的偏移
int main()
{
    int a[N]={1,2,3,4,5};
    int *p;
    int i;
    p=a; //保证等号两边的数值类型一致
    for(i=0;i<N;i++) //正序输出
    {
        printf("%3d",*(p+i));
    }
    printf("\n-----\n");
    p=&a[4]; //让 p 指向最后一个元素
    for(i=0;i<N;i++) //逆序输出
    {
        printf("%3d",*(p-i));
    }
    printf("\n");
    return 0;
}
```

如下图所示，数组名中存储着数组的起始地址 0x61fdf0，其类型为整型指针，所以可以将其赋值给整型指针变量 p，可以从监视窗口中看到 p+1 的值为 0x61fdf4。那么为什么加 1 后不是 0x61fdf1 呢？因为指针变量加 1 后，偏移的长度是其基类型的长度，也就是偏移 sizeof(int)，这样通过*(p+1)就可以得到元素 a[1]。编译器在编译时，数组取下标的操作正是转换为指针偏移来完成

的。

```
5 int main()
6 {
7     int a[N]={1,2,3,4,5};  a: int [5]
8     int *p;  p: 0x61fdf0
9     int i;  i: 0
10    p=a; //保证等号两边的数值类型一致 p: 0x61fdf0  a: int [5]
11    for(i=0;i<N;i++) //正序输出 i: 0
12    {
13        printf(_Format: "%3d",*(p+i));
14    }
15 }
```

控制台中显示内存视图：

```
&a = (int (*)[5] | 0x61fdf0) 0x61fdf0
p+1 = (int * | 0x61fdf4) 0x61fdf4
a = (int [5])
p = (int * | 0x61fdf0) 0x61fdf0
```

红色箭头指向地址 0x61fdf0，并附有注释：“这是手动输入的，&a和a中存储的地址值是一致的”。

图 1 例 1.1 中代码的执行效果

2 指针与一维数组

为什么一维数组在函数调用进行传递时，它的长度函数无法知道呢？这是由于一维数组名中存储的是数组的首地址。如下例所示，数组名 `c` 中存储的是一个起始地址（这里最好结合本小节视频来学习，才能理解更加清晰），所以子函数 `change` 中其实传入了一个地址。定义一个指针变量时，指针变量的类型要和数组的数据类型保持一致，通过取值操作，就可将“`h`”改为“`H`”，这种方法称为指针法。获取数组元素时，也可以通过取下标的方式来获取数组元素并进行修改，这种方法称为下标法。

【例】数组传递给子函数的实战练习

```
#include <stdio.h>
```

```
//数组名作为实参传递给子函数时，是弱化为指针的
```

```
//练习传递与偏移
```

```
void change(char *c)
```

```
{
    *c = 'H';
    c[1] = 'E';
    *(c + 2) = 'L';
}
```

```
int main()
```

```
{
    char c[10] = "hello";
    change(c);
}
```

```
puts(c);  
return 0;  
}
```